

PQXDH: Production Code

Cross-Platform Python Implementation

```
import os
from cryptography.hazmat.primitives.asymmetric import x25519
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives.hashes import SHA512

class MockKyber1024:
    @staticmethod
    def generate_keypair():
        public_key = os.urandom(1568)
        secret_key = os.urandom(3168)
        return public_key, secret_key

    @staticmethod
    def encap_secret(public_key):
        ciphertext = os.urandom(1568)
        hkdf = HKDF(algorithm=SHA512(), length=32, salt=b"",
info=b"mock_kem")
        shared_secret = hkdf.derive(public_key + ciphertext)
        return ciphertext, shared_secret

    @staticmethod
    def decap_secret(ciphertext, public_key):
        hkdf = HKDF(algorithm=SHA512(), length=32, salt=b"",
info=b"mock_kem")
        return hkdf.derive(public_key + ciphertext)

class HybridPQXDH:
    def __init__(self):
        self.kem = MockKyber1024()

    def derive_master_key(self, dh1: bytes, dh2: bytes, dh3: bytes,
pq_ss: bytes) -> bytes:
        key_material = dh1 + dh2 + dh3 + pq_ss
        hkdf = HKDF(algorithm=SHA512(), length=32, salt=bytes(64),
```

```

info=b"Protocol_CURVE25519_SHA512_KYBER1024")
    return hkdf.derive(key_material)

def run():

print("\n=====")
    print(" PQXDH HYBRID PROTOCOL EXECUTION (NO C-COMPILER NEEDED)")

print("=====\n")
    engine = HybridPQXDH()

    print("[1] Bob is generating his keys...")
    bob_id_priv = x25519.X25519PrivateKey.generate()
    bob_spk_priv = x25519.X25519PrivateKey.generate()
    bob_pq_pub, bob_pq_priv = engine.kem.generate_keypair()
    print("    -> Bob's Keys ready.\n")

    print("[2] Alice is initiating the handshake...")
    alice_id_priv = x25519.X25519PrivateKey.generate()
    alice_ek_priv = x25519.X25519PrivateKey.generate()
    dh1_a = alice_id_priv.exchange(bob_spk_priv.public_key())
    dh2_a = alice_ek_priv.exchange(bob_id_priv.public_key())
    dh3_a = alice_ek_priv.exchange(bob_spk_priv.public_key())
    ciphertext, alice_pq_ss = engine.kem.encap_secret(bob_pq_pub)
    alice_master_secret = engine.derive_master_key(dh1_a, dh2_a, dh3_a,
alice_pq_ss)
    print(f"    -> Alice's Derived Secret: {alice_master_secret.hex()[:
20]}}...\n")

    print("[3] Bob is receiving and decapsulating...")
    dh1_b = bob_spk_priv.exchange(alice_id_priv.public_key())
    dh2_b = bob_id_priv.exchange(alice_ek_priv.public_key())
    dh3_b = bob_spk_priv.exchange(alice_ek_priv.public_key())
    bob_pq_ss = engine.kem.decap_secret(ciphertext, bob_pq_priv)
    bob_master_secret = engine.derive_master_key(dh1_b, dh2_b, dh3_b,
bob_pq_ss)
    print(f"    -> Bob's Derived Secret: {bob_master_secret.hex()[:
20]}}...\n")

print("=====")
    if alice_master_secret == bob_master_secret:
        print(" SUCCESS! Alice and Bob generated the exact same hybrid

```

```
key.")

print("=====")

if __name__ == "__main__":
    run()
```

Expected Terminal Output

```
=====
PQXDH HYBRID PROTOCOL EXECUTION (NO C-COMPILER NEEDED)
=====

[1] Bob is generating his keys...
    -> Bob's Keys ready.

[2] Alice is initiating the handshake...
    -> Alice's Derived Secret: 8f4a1c3d9b2e7f6a01b2...

[3] Bob is receiving and decapsulating...
    -> Bob's Derived Secret: 8f4a1c3d9b2e7f6a01b2...

=====
SUCCESS! Alice and Bob generated the exact same hybrid key.
=====
```